



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Dancel, Jamella Tracy L.
Section:	AN42
Professor:	Ms. Crisola G. Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects
 - If-Else statements
 - Loops (for loop)

- Functions / methods
- 2. Understand the environment:
 - Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python38-64\python.exe
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:

- The agent's current location
- The action taken (cleaned/moved)
- Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |
```

Your Code - Laboratory Tasks 

```

import random
import time

# =====
# TASK 1: Environment Setup
# =====

def is_room_dirty(environment, room_name):
    """Check if a given room is dirty."""
    return environment.get(room_name, "").lower() == "dirty"

def clean_room(environment, room_name):
    """Clean the specified room."""
    environment[room_name] = "Clean"

# =====
# TASK 2: Rule-Based Vacuum Agent Implementation
# =====

class VacuumAgent:
    def __init__(self, current_location="A"):
        self.current_location = current_location

    def move_2_rooms(self):
        """Move method for Task 2 (2-room environment)."""
        if self.current_location == "A":
            self.current_location = "B"
        else:
            self.current_location = "A"

    def move_3_rooms(self, environment):
        """
        Move method for Bonus Task (3-room environment).
        Prioritizes moving to a random dirty room; otherwise selects any other room.
        """
        other_rooms = [r for r in environment.keys() if r != self.current_location]
        dirty_rooms = [r for r in other_rooms if is_room_dirty(environment, r)]

        if dirty_rooms:
            self.current_location = random.choice(dirty_rooms)
        else:

```

```

        self.current_location = random.choice(other_rooms)

def perceive_and_act(self, environment, is_three_rooms=False):
    """
    Perceive state and apply rules:
    Rule 1: If current room is dirty, clean it.
    Rule 2: If current room is clean, move to another room.
    """
    curr_room = self.current_location

    if is_room_dirty(environment, curr_room):
        clean_room(environment, curr_room)
        action = f"Cleaned room {curr_room}"
    else:
        if is_three_rooms:
            self.move_3_rooms(environment)
        else:
            self.move_2_rooms()
        action = f"Moved to room {self.current_location}"

    return action

# =====
# BONUS TASK: Visualizer
# =====

def render_grid(environment, agent_location):
    """Renders a text-based visual grid of the environment and agent position."""
    rooms = list(environment.keys())
    top_border = "+" + "-----+" * len(rooms)

    header_row = "|"
    status_row = "|"
    agent_row = "|"

    for r in rooms:
        header_row += f" Room {r:2s} |"
        status_row += f" {environment[r]:5s} |"
        agent_marker = "[ AGENT ]" if r == agent_location else "    "
        agent_row += f" {agent_marker} |"

    print(top_border)

```

```

print(header_row)
print(status_row)
print(agent_row)
print(top_border)

# =====
# TASK 3 & BONUS TASK: Run Simulations
# =====

def run_standard_simulation():
    print("="*50)
    print("  STANDARD SIMULATION (2 ROOMS: A & B)  ")
    print("="*50)

    # 1. Environment Setup
    state_a = input("Enter initial state for Room A (Dirty/Clean): ").strip().capitalize()
    state_b = input("Enter initial state for Room B (Dirty/Clean): ").strip().capitalize()

    environment = {
        "A": state_a if state_a in ["Dirty", "Clean"] else "Dirty",
        "B": state_b if state_b in ["Dirty", "Clean"] else "Dirty"
    }

    start_loc = input("Enter starting room for Agent (A/B): ").strip().upper()
    if start_loc not in ["A", "B"]:
        start_loc = "A"

    steps = int(input("Enter number of simulation steps: "))

    agent = VacuumAgent(current_location=start_loc)

    print("\nStarting 2-Room Simulation...")
    print(f"Initial State: {environment}\n")

    for step in range(1, steps + 1):
        action = agent.perceive_and_act(environment, is_three_rooms=False)
        print(f"Step {step}:")
        print(f" Agent Location: Room {agent.current_location}")
        print(f" Action Taken : {action}")
        print(f" Env State   : {environment}")
        print("-" * 40)

```

```

def run_bonus_simulation():
    print("\n" + "="*50)
    print("    BONUS SIMULATION (3 ROOMS: A, B & C)    ")
    print("="*50)

    state_a = input("Enter initial state for Room A (Dirty/Clean): ").strip().capitalize()
    state_b = input("Enter initial state for Room B (Dirty/Clean): ").strip().capitalize()
    state_c = input("Enter initial state for Room C (Dirty/Clean): ").strip().capitalize()

    environment = {
        "A": state_a if state_a in ["Dirty", "Clean"] else "Dirty",
        "B": state_b if state_b in ["Dirty", "Clean"] else "Dirty",
        "C": state_c if state_c in ["Dirty", "Clean"] else "Dirty"
    }

    start_loc = input("Enter starting room for Agent (A/B/C): ").strip().upper()
    if start_loc not in ["A", "B", "C"]:
        start_loc = "A"

    steps = int(input("Enter number of simulation steps: "))

    agent = VacuumAgent(current_location=start_loc)

    print("\nStarting 3-Room Visual Simulation...")
    render_grid(environment, agent.current_location)

    for step in range(1, steps + 1):
        time.sleep(0.5) # Brief pause for step display
        action = agent.perceive_and_act(environment, is_three_rooms=True)
        print(f"\nStep {step}: {action}")
        render_grid(environment, agent.current_location)

# =====
# MAIN EXECUTION
# =====

if __name__ == "__main__":
    run_standard_simulation()

    run_bonus = input("\nDo you want to run the Bonus Task (3 Rooms with Grid Visuals)? (y/n): ").strip().lower()
    if run_bonus == 'y':

```

run_bonus_simulation()

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. What rules does the agent follow?

Rule 1: If the current room is perceived as **Dirty**, execute the **Clean** action.

Rule 2: If the current room is perceived as **Clean**, execute the **Move** action to shift to another room.

2. How does the agent decide when to clean and when to move?

The agent decides purely on condition-action matching (reflex mechanism) based on its current location sensor input. It checks the condition status (**Dirty** vs **Clean**) of the room it currently occupies. If **Dirty**, it cleans; if **Clean**, it transitions location.

3. What happens if both rooms are already clean at the start?

The agent perceives the starting room as **Clean** and immediately invokes **Rule 2** to move to the other room. In subsequent steps, because all rooms remain clean, the agent continuously loops moving back and forth between the clean rooms indefinitely until the total execution steps exhaust.

4. How would you modify the agent to handle more than two rooms?

State Expansion: Expand the environment state representation (e.g., dictionary keys) to accommodate additional room identifiers (**Room C**, **Room D**, etc.).

Movement Logic: Replace simple binary toggles (if **A** then **B**) with dynamic routing choices (e.g., prioritizing uncleaned/dirty adjacent rooms via pathfinding or random choices among remaining dirty rooms, as implemented in the bonus module above).